UNIT KOMPETENSI 08 • TI.LB.08.2026

MENGGUNAKAN LIBRARY ATAU KOMPONEN PRE-EXISTING

Implementasi Riil pada Proyek: Portal Karir, Profil Industri & Modul Automation-GPT (Fullstack Web)

Presenter / Mahasiswa	Aditya Ristanto Mukti (Fullstack Web Developer Intern)	**Repository**	`github.com/AdityaRistantoMukti/Website_Intersis`
Institusi / Mitra	PT Global Philsonic Technologies (Intersis) Batam	**Tech Stack Utama**	React 19, Vite, Tailwind CSS v4, Node.js Express 5, MySQL2
Cakupan Outline Sub-Title	Identifikasi unit reuse • Integrasi library eksternal (npm) • Pembaruan versi library		

I. PETA OUTLINE & STRUKTUR PEMBAHASAN

Unit kompetensi **Menggunakan Library atau Komponen Pre-Existing** ini disusun secara berkesinambungan mencakup seluruh sub-judul yang didefinisikan pada diagram alur kompetensi, diaplikasikan langsung pada arsitektur website yang telah dibangun:

- **Identifikasi unit reuse**: Menganalisis kebutuhan perangkat lunak dan menentukan komponen siap pakai terverifikasi (Build vs Buy/Use Analysis).
- **Integrasi library eksternal (npm)**: Menginstalasi dan mengonfigurasi pustaka kunci di backend (Express, JWT, Bcrypt, Multer) dan frontend (Three.js, Cobe, i18next).
- **Pembaruan versi library**: Mengelola semantic versioning (^ vs ~), penguncian dependensi via package-lock.json, dan mitigasi breaking changes.

II. NASKAH WICARA LENGKAP PRESENTER (VERBATIM SPEECH SCRIPT)

🎤 PANDUAN NARASI WICARA PRESENTER (SPEAKING SCRIPT)

[Pembukaan & Sikap] "Dewan Penguji yang saya hormati, dalam rekayasa perangkat lunak modern, kecepatan dan keandalan sistem sangat didukung oleh kemampuan memilih serta mengintegrasikan pustaka kode yang sudah teruji. Pada unit kompetensi **Menggunakan Library atau Komponen Pre-Existing**, saya akan memaparkan bagaimana ekosistem open-source dimanfaatkan secara optimal pada proyek ini."

[Menjelaskan Sub 1: Identifikasi Unit Reuse] "Langkah awal kami adalah *Identifikasi Unit Reuse* melalui analisis 'Build vs Buy/Use'. Kami menyadari bahwa membangun sistem kriptografi autentikasi sendiri atau menulis algoritma WebGL 3D dari nol adalah langkah yang rentan celah keamanan dan membuang waktu. Oleh karena itu, kami menetapkan unit-unit kritis yang wajib menggunakan library standar industri berlisensi resmi: seperti `jsonwebtoken` untuk token stateless, `bcrypt` untuk hashing, `lucide-react` untuk ikon SVG, dan `cobe` untuk animasi bola dunia interaktif."

[Menjelaskan Sub 2: Integrasi Library Eksternal via NPM] "Pada sub-topik *Integrasi Library Eksternal*, kami mengelola seluruh pustaka menggunakan **Node Package Manager (NPM)**. Di backend, kami mengintegrasikan `mysql2/promise` untuk koneksi database, `multer` untuk upload media, dan `helmet` untuk injeksi security headers. Di frontend, kami memasang `react-router-dom` untuk client-side routing, `axios` untuk HTTP networking, serta `sweetalert2` untuk dialog antarmuka yang elegan. Seluruh integrasi dikonfigurasi melalui berkas inisialisasi terpusat agar tidak mencemari komponen lain."

[Menjelaskan Sub 3: Pembaruan Versi Library] "Pilar ketiga adalah *Pembaruan Versi Library*. Kami memahami prinsip **Semantic Versioning (SemVer)** yang membedakan kenaikan Major, Minor, dan Patch version. Untuk menjamin konsistensi build antar lingkungan pengembang dan server produksi (Railway & Vercel), kami selalu menyertakan berkas `package-lock.json` di dalam version control Git. Kami juga rutin menjalankan perintah `npm audit` untuk memindai kerentanan keamanan dan memperbarui versi pustaka secara terkontrol tanpa merusak kompatibilitas fitur yang sudah ada."

III. PEMBAHASAN TEKNIS RINCI & BEDAH KODE NYATA PROYEK

Berikut adalah pembuktian implementasi teknis dari masing-masing sub-judul berdasarkan baris kode dan berkas nyata di dalam repository proyek:

8.1. Analisis Identifikasi Komponen Pre-Existing (Build vs Use)

Kaitkan ke Codebase: **`Dokumentasi Arsitektur Eksternal Library`**

Daftar identifikasi kebutuhan fungsional dan pemilihan pustaka pre-existing yang diadopsi:

Fungsionalitas Sistem	Pustaka Dipilih	Alasan & Nilai Tambah Penggunaan
Autentikasi & Keamanan	`jsonwebtoken` & `bcrypt`	Standar de facto industri untuk keamanan stateless token JWT dan algoritma hashing Blowfish yang tahan serangan brute-force.
Visualisasi 3D Interaktif	`cobe` & `three`	Menyediakan canvas WebGL 60 FPS berbobot sangat ringan (~5KB gzipped untuk Cobe) dibanding memuat 3D engine utuh.
Multilingual (ID / EN)	`react-i18next`	Pengelolaan kamus bahasa berbasis JSON yang modular dengan deteksi otomatis preferensi bahasa peramban web.
Ikonografi Vektor	`lucide-react`	Ratusan ikon SVG yang tree-shakeable, ringan, dan dapat dikontrol warna serta ukurannya secara dinamis via props React.

8.2. Integrasi Library Kunci di Backend & Frontend via NPM

Kaitkan ke Codebase: **`backend/package.json & frontend/src/i18n.js`**

Konfigurasi integrasi library multi-bahasa `i18next` pada sisi client:

frontend/src/i18n.js

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import LanguageDetector from 'i18next-browser-languagedetector';
import translationID from './locales/id.json';
import translationEN from './locales/en.json';

// Inisialisasi Integrasi Eksternal Library i18next
i18n
  .use(LanguageDetector)
  .use(initReactI18next)
  .init({
    resources: {
      id: { translation: translationID },
      en: { translation: translationEN }
    },
    fallbackLng: 'id',
    interpolation: { escapeValue: false }
  });

export default i18n;
```

8.3. Manajemen Semantic Versioning & Penguncian package-lock.json

Kaitkan ke Codebase: **`package-lock.json & npm audit workflow`**

Penerapan aturan versioning untuk mencegah terjadinya build break pada environment produksi:

- **Caret Operator (^):** Digunakan untuk dependensi yang aman menerima update Minor dan Patch (seperti `"axios": "^1.20.0"`).

- **Exact Locking:** Diterapkan pada library grafis sensitif seperti `"cobe": "0.6.3"` untuk mencegah perubahan shader pada update minor.
- **Integritas Hash `package-lock.json`:** Menjamin setiap mesin pengembang dan runner Railway menginstalasi dependensi dengan checksum SHA-512 yang sama persis.

```
Terminal Command: Audit & Versioning Verification
# Memeriksa dependensi yang memiliki versi baru
npm outdated

# Memindai potensi celah keamanan (vulnerabilities)
npm audit

# Menginstalasi versi terkunci persis sesuai package-lock.json di CI/CD
npm ci
```

IV. POIN KUNCI PRESENTASI (SLIDE TALKING POINTS)

POINTER RINGKAS UNTUK TAMPILAN SLIDE / LAYAR

- Prinsip 'Build vs Use': Memilih library teruji untuk menghemat waktu dan meningkatkan reliabilitas.
- Backend Ecosystem: Express 5, MySQL2, JWT, Bcrypt, Multer, Helmet, dan Compression.
- Frontend Ecosystem: React 19, Vite, Tailwind v4, Lucide Icons, Cobe 3D Globe, dan i18next.
- Semantic Versioning (SemVer): Pengendalian update Major, Minor, dan Patch secara disiplin.
- Konsistensi Lingkungan: Penguncian dependensi menggunakan package-lock.json dan verifikasi npm audit.

V. MATRIKS TANYA JAWAB & PANDUAN PEMBELAAN SIDANG (Q&A DEFENSE)

Prediksi pertanyaan evaluasi dari Penguji / Asesor / Pembimbing Industri beserta jawaban teknis yang siap dipertahankan:

Q1: Apa risiko jika kita tidak mengikutsertakan file package-lock.json ke dalam repositori Git?

Jawaban Pembelaan Presenter: Tanpa package-lock.json, setiap kali perintah npm install dijalankan di server produksi atau komputer tim lain, npm dapat mengunduh versi library terbaru yang berbeda dengan versi yang digunakan saat development. Jika versi baru tersebut mengandung bug atau breaking change, aplikasi produksi dapat langsung crash.

Q2: Mengapa Anda menggunakan 'npm ci' bukan 'npm install' pada skrip deployment server?

Jawaban Pembelaan Presenter: 'npm ci' (Clean Install) dirancang khusus untuk lingkungan otomatisasi dan server produksi. Perintah ini menghapus node_modules lama dan menginstalasi dependensi secara cepat dan persis 100% mengikuti checksum yang tertulis di package-lock.json, serta menolak instalasi jika package.json dan package-lock.json tidak sinkron.